**Universidad Autónoma de Madrid**
**Escuela Politécnica Superior**

**Grado en Ciencia e Ingeniería de Datos**

**Proyecto de la asignatura**
**Aprendizaje Automático III**

# Sistema Generativo basado en Modelos de Difusión

Realizado por
Eva Blázquez y Gabriela Damas

Propuesto por
Prof. Alberto Suárez González

Abril 2026

# Código Fuente

El código fuente, los notebooks experimentales y los modelos entrenados están disponibles en el siguiente repositorio:

`https://github.com/gabrieladamasuam/DiffusionModels`

# Resumen

En este proyecto se desarrolla un sistema de inteligencia artificial generativa basado en modelos de difusión para la generación y análisis de imágenes. El sistema se ha diseñado de forma modular y configurable, permitiendo seleccionar distintos procesos de difusión, esquemas de ruido y métodos de muestreo.

Se implementan dos formulaciones continuas de modelos de difusión: los modelos de varianza creciente (*Variance Exploding*, VE), basados en movimiento browniano, y los modelos de varianza preservada (*Variance Preserving*, VP), basados en el proceso de Ornstein–Uhlenbeck. El sistema incorpora distintos métodos de generación, incluyendo el integrador de Euler–Maruyama, esquemas predictor–corrector y la formulación determinista basada en la *probability flow ODE*. Asimismo, se consideran diferentes esquemas de ruido, como los perfiles *linear*, *cosine* y *sigmoid*.

El modelo de score se entrena mediante *denoising score matching* utilizando arquitecturas tipo U-Net con embeddings temporales basados en *Gaussian random Fourier features*. La validación experimental se realiza principalmente sobre MNIST, restringido al dígito 3, lo que permite comparar de forma controlada las distintas configuraciones implementadas.

En primer lugar, se comparan los modelos VE y VP junto con los distintos samplers y esquemas de ruido. A partir de esta comparación se selecciona la configuración más estable y con mejor comportamiento experimental. Posteriormente, el sistema se extiende a tareas de generación controlable, incluyendo condicionamiento por clase y color, así como imputación de regiones.

La evaluación se realiza mediante métricas estándar como FID, IS y BPD, complementadas con inspección visual de las muestras generadas. En particular, la BPD se calcula mediante estimación de la log-verosimilitud usando la *probability flow ODE*. Además, se incluye en el apéndice una exploración preliminar sobre AFHQ como prueba de escalabilidad a imágenes RGB naturales, sin plantearla como resultado competitivo.

Los resultados obtenidos muestran que la configuración VP con schedule *cosine* y sampler predictor–corrector ofrece el mejor comportamiento en MNIST, y permiten analizar el impacto de los distintos componentes del sistema desarrollado.

# Índice general

**1. Introducción** **1**
1.1. Motivación y contexto . . . . . 1
1.2. Objetivos del proyecto . . . . . 1
1.3. Alcance del trabajo . . . . . 2
1.4. Organización del documento . . . . . 2

**2. Estado del arte** **3**
2.1. Notación . . . . . 3
2.2. Modelos de difusión . . . . . 3
2.2.1. Variance Exploding (VE) . . . . . 4
2.2.2. Variance Preserving (VP) . . . . . 5
2.3. Score-based models . . . . . 6
2.4. Arquitectura U-Net . . . . . 7
2.5. Métodos de muestreo . . . . . 7
2.5.1. Métodos predictor–corrector . . . . . 8
2.5.2. Probability Flow ODE . . . . . 8
2.6. Métricas de evaluación . . . . . 9
2.6.1. Fréchet Inception Distance (FID) . . . . . 9
2.6.2. Inception Score (IS) . . . . . 10
2.6.3. Bits Per Dimension (BPD) . . . . . 10

**3. Desarrollo de software** **11**
3.1. Análisis de Requisitos . . . . . 11
3.1.1. Requisitos funcionales . . . . . 11
3.1.2. Requisitos no funcionales . . . . . 11
3.2. Casos de Uso . . . . . 12
3.3. Arquitectura del sistema . . . . . 12
3.4. Validación y calidad del software . . . . . 14
3.5. Planificación del proyecto . . . . . 14

**4. Experimentación y resultados** **15**
4.1. Configuración experimental . . . . . 15
4.2. Evaluación del modelo VE . . . . . 16
4.3. Evaluación del modelo VP . . . . . 17
4.4. Comparación final entre VE y VP . . . . . 18
4.5. Generación condicional de imágenes a color . . . . . 19

4.6. Imputing . . . . . 20

**5. Conclusiones** **21**

**I. Derivaciones teóricas** **23**

I.1. Derivación de la ecuación de Fokker–Planck . . . . . 23

I.2. Solución marginal del proceso Browniano VE . . . . . 25

I.3. Solución marginal del proceso Ornstein–Uhlenbeck VP . . . . . 27

I.4. Derivación de la pérdida de denoising score matching . . . . . 30

I.5. Cálculo de BPD mediante la probability flow ODE . . . . . 32

**II. Gráficas complementarias** **35**

II.1. Gráficas de VE . . . . . 35

II.2. Gráficas de VP . . . . . 36

II.3. Gráficas comparativas entre VE y VP . . . . . 37

**III. Generación de imágenes en AFHQ** **38**

**IV. Imágenes generadas adicionales** **42**

# Capítulo 1. Introducción

## 1.1. Motivación y contexto

La inteligencia artificial generativa ha experimentado un crecimiento notable en los últimos años, especialmente en el ámbito de la generación de imágenes. Modelos como las GANs (*Generative Adversarial Networks*) y los VAEs (*Variational Autoencoders*) han permitido avances significativos, aunque presentan limitaciones en términos de estabilidad de entrenamiento y calidad de las muestras generadas.

En este contexto, los modelos de difusión han emergido como una de las aproximaciones más robustas y eficaces. Este enfoque permite generar muestras de alta calidad a partir de distribuciones simples, mediante modelos probabilísticos bien fundamentados que modelan explícitamente el proceso de generación.

## 1.2. Objetivos del proyecto

El objetivo de este proyecto es abordar el problema de la generación de imágenes mediante modelos de difusión en tiempo continuo, diseñando e implementando un sistema generativo completo que permita estudiar el comportamiento de distintas variantes del proceso de difusión.

Además, el problema no se limita únicamente a la generación de imágenes, sino que incluye la evaluación sistemática de los distintos componentes del sistema, como los esquemas de ruido, los métodos de muestreo y las métricas de calidad. Asimismo, se considera la extensión del modelo hacia mecanismos de generación controlable.

Como objetivos específicos se plantean:

- Implementar modelos de difusión VE y VP en un marco continuo basado en ecuaciones diferenciales estocásticas (SDEs).
- Diseñar y entrenar modelos de score utilizando arquitecturas tipo U-Net.
- Implementar distintos métodos de muestreo, incluyendo Euler–Maruyama, predictor–corrector y probability flow ODE.
- Analizar el impacto de distintos esquemas de ruido (linear, cosine y sigmoid).
- Evaluar la calidad de las muestras generadas mediante métricas estándar como FID, IS y BPD.

- Incorporar mecanismos de generación controlable, como el condicionamiento por clase y atributos.
- Explorar técnicas de imputación de imágenes mediante modelos de difusión.
- Validar el sistema en datasets de distinta complejidad, utilizando MNIST como escenario principal de evaluación y AFHQ como prueba adicional de escalabilidad.

## 1.3. Alcance del trabajo

El sistema desarrollado se centra en la generación de imágenes en dos escenarios diferenciados. En primer lugar, se utiliza el dataset MNIST como un entorno controlado que permite validar la correcta implementación del modelo y del proceso de difusión.

En segundo lugar, se realiza una exploración preliminar sobre el dataset AFHQ, centrado en imágenes de gatos, con el objetivo de comprobar la escalabilidad del pipeline a imágenes RGB naturales. Debido a la mayor complejidad visual de este dataset y a las limitaciones de tiempo de entrenamiento, estos resultados se presentan en el apéndice y se interpretan de forma cualitativa, no como una comparación exhaustiva ni como un resultado competitivo.

El trabajo incluye tanto modelos no condicionados como extensiones hacia la generación condicional, permitiendo controlar atributos como la clase o características adicionales de las imágenes. Asimismo, se consideran arquitecturas basadas en U-Net y métodos de muestreo derivados de la integración numérica de ecuaciones diferenciales estocásticas y ordinarias.

## 1.4. Organización del documento

El resto del documento se estructura de la siguiente manera:

- En el Capítulo 2 se presenta el estado del arte sobre modelos de difusión, incluyendo su formulación matemática y las principales variantes VE y VP.
- En el Capítulo 3 se describe el desarrollo del sistema, incluyendo el diseño del software, los módulos implementados y los requisitos del sistema.
- En el Capítulo 4 se muestran los resultados experimentales y su análisis comparativo.
- En el Capítulo 5 se exponen las conclusiones y posibles líneas de trabajo futuro.

Los apéndices recogen las derivaciones teóricas, las gráficas complementarias y una exploración adicional sobre la generación de imágenes RGB en AFHQ.

# Capítulo 2. Estado del arte

## 2.1. Notación

Sea $\mathbf{x}_0 \in \mathbb{R}^D$ una imagen vectorizada, donde $D$ representa el número total de dimensiones, es decir, el producto entre altura, anchura y número de canales. Denotamos por $p_0(\mathbf{x})$ la distribución desconocida de los datos reales y por $p_t(\mathbf{x})$ la distribución de las muestras perturbadas en el instante $t \in [0, T]$.

El objetivo de un modelo generativo es obtener nuevas muestras $\mathbf{x}_0 \sim p_0(\mathbf{x})$ a partir de una distribución simple $\pi_T(\mathbf{x})$, normalmente gaussiana, de la cual es fácil muestrear.

## 2.2. Modelos de difusión

Los modelos de difusión constituyen una familia de modelos generativos probabilísticos basados en los principios de la termodinámica de no equilibrio. Su objetivo es aprender la distribución de una variable compleja mediante un proceso de difusión progresivo en el que los datos se degradan mediante la adición de ruido.

En el proceso directo (*forward*), se añade ruido gaussiano de forma progresiva a las muestras a lo largo del intervalo $t \in [0, T]$.

En tiempo continuo, el proceso directo se modela mediante una ecuación diferencial estocástica (SDE) de Itô:

$$\mathrm{d}\mathbf{x}(t) = f(\mathbf{x}(t), t)\,\mathrm{d}t + g(t)\,\mathrm{d}\mathbf{W}(t), \tag{2.1}$$

donde $f(\mathbf{x}, t)$ es el término de deriva, $g(t)$ es el coeficiente de difusión y $\mathbf{W}(t)$ es un proceso de Wiener estándar. El primer término describe la evolución determinista media del sistema, mientras que el segundo introduce perturbaciones aleatorias gaussianas.

Si $\mathbf{x}(0) \sim p_0(\mathbf{x})$, la SDE induce una familia de distribuciones marginales $\{p_t(\mathbf{x})\}_{t\in[0,T]}$. El proceso se diseña de forma que, para $t = T$, la distribución $p_T(\mathbf{x})$ sea próxima a una distribución de referencia $\pi_T(\mathbf{x})$, normalmente una normal multivariante.

La evolución de estas densidades marginales puede describirse mediante la ecuación de Fokker–Planck. En el caso en que el coeficiente de difusión no depende de $\mathbf{x}$, se obtiene:

$$\frac{\partial p_t(\mathbf{x})}{\partial t} = -\nabla_{\mathbf{x}} \cdot (f(\mathbf{x}, t)p_t(\mathbf{x})) + \frac{1}{2}g(t)^2 \Delta_{\mathbf{x}} p_t(\mathbf{x}), \tag{2.2}$$

donde $\Delta_{\mathbf{x}}$ denota el operador laplaciano. Esta ecuación no describe trayectorias individuales, sino la evolución temporal de la densidad de probabilidad asociada al proceso.

La derivación formal de esta ecuación a partir de la SDE se recoge en el Apéndice I.1.

La clave de estos modelos es el aprendizaje del proceso inverso (*backward*). A diferencia de los sistemas físicos naturales, donde la flecha del tiempo dicta que el desorden es irreversible, estos modelos entrenan una red neuronal para aprender la dinámica temporal inversa. Al estimar el gradiente de la densidad de los datos en cada nivel de ruido —técnicamente conocido como la función de score— el modelo adquiere la capacidad de revertir la difusión.

Este enfoque ha demostrado ser altamente eficaz en la generación de imágenes, alcanzando resultados comparables o superiores a otros modelos generativos, como GANs o VAEs.

Dentro del marco de modelos de difusión continuos, existen distintas formulaciones en función de cómo evoluciona la varianza del proceso a lo largo del tiempo. En este proyecto se consideran dos de las más relevantes:

### 2.2.1. Variance Exploding (VE)

En estos modelos, la media se mantiene centrada en la muestra original, mientras que la varianza del proceso aumenta progresivamente con el tiempo, lo que justifica el término *Variance Exploding*. Este comportamiento está determinado por un coeficiente de difusión creciente $g(t)$.

$$\mathrm{d}\mathbf{x}(t) = g(t)\,\mathrm{d}\mathbf{W}(t), \tag{2.3}$$

Integrando entre 0 y $t$, se obtiene:

$$\mathbf{x}(t) = \mathbf{x}_0 + \int_0^t g(s)\,\mathrm{d}\mathbf{W}(s). \tag{2.4}$$

Dado que el término estocástico es gaussiano de media nula, la distribución condicionada a la muestra inicial viene dada por:

$$p_{0t}(\mathbf{x}(t) \mid \mathbf{x}_0) = \mathcal{N}\left(\mathbf{x}(t); \mathbf{x}_0, \sigma^2(t)\mathbf{I}\right), \tag{2.5}$$

donde

$$\sigma^2(t) = \int_0^t g^2(s)\,\mathrm{d}s. \tag{2.6}$$

### 2.2.2. Variance Preserving (VP)

En estos modelos, la varianza del proceso permanece acotada gracias a la presencia de un término de deriva que compensa la difusión, lo que justifica el término *Variance Preserving*. Este comportamiento viene determinado por una función $\beta(t)$ que controla la tasa de ruido.

$$\mathrm{d}\mathbf{x}(t) = -\frac{1}{2}\beta(t)\mathbf{x}(t)\,\mathrm{d}t + \sqrt{\beta(t)}\,\mathrm{d}\mathbf{W}(t), \tag{2.7}$$

Esta formulación es equivalente a un proceso de Ornstein–Uhlenbeck con coeficientes dependientes del tiempo. La solución puede escribirse como:

$$\mathbf{x}(t) = \alpha(t)\mathbf{x}_0 + \sigma(t)\mathbf{z}, \qquad \mathbf{z} \sim \mathcal{N}(0, \mathbf{I}), \tag{2.8}$$

donde

$$\alpha(t) = \exp\left(-\frac{1}{2}\int_0^t \beta(s)\,\mathrm{d}s\right), \qquad \sigma^2(t) = 1 - \exp\left(-\int_0^t \beta(s)\,\mathrm{d}s\right). \tag{2.9}$$

Por tanto, condicionado a la muestra original, la distribución es gaussiana:

$$p_{0t}(\mathbf{x}(t) \mid \mathbf{x}_0) = \mathcal{N}\left(\mathbf{x}(t); \alpha(t)\mathbf{x}_0, \sigma^2(t)\mathbf{I}\right). \tag{2.10}$$

A diferencia del caso VE, la media del proceso decae progresivamente hacia cero debido al término de deriva, mientras que la varianza permanece acotada. Esta propiedad permite controlar mejor la dispersión del ruido y favorece la estabilidad del entrenamiento.

Ambas formulaciones comparten la misma base en procesos de difusión y en el aprendizaje del score, pero difieren principalmente en el comportamiento de la varianza:

- En los modelos VE, la varianza crece sin cota superior, lo que permite explorar regiones amplias del espacio de estados, aunque puede dificultar la estabilidad numérica.
- En los modelos VP, la varianza permanece acotada debido al término de deriva, lo que favorece una dinámica más estable y una mejor conservación de la estructura de los datos.

En la práctica, los modelos VE tienden a generar mayor diversidad en las muestras, mientras que los modelos VP proporcionan una mayor estabilidad durante el entrenamiento y el muestreo.

En este trabajo, ambas formulaciones se implementan y comparan experimentalmente para analizar su impacto en la generación de imágenes.

## 2.3. Score-based models

Los modelos basados en score constituyen el núcleo teórico de los modelos de difusión modernos. En lugar de modelar directamente la densidad de probabilidad de los datos, estos modelos aprenden el gradiente del logaritmo de dicha densidad, conocido como *score*:

$$s_\theta(\mathbf{x}, t) \approx \nabla_{\mathbf{x}} \log p_t(\mathbf{x}), \tag{2.11}$$

donde $p_t(\mathbf{x})$ denota la distribución de los datos perturbados con ruido en el instante $t$.

Este gradiente indica la dirección en la que se debe modificar una muestra para aumentar su probabilidad bajo la distribución objetivo.

El entrenamiento se realiza mediante *denoising score matching*, en el que se añaden perturbaciones gaussianas a las muestras reales y se entrena una red neuronal para aproximar la función de score de las distribuciones ruidosas. Para ello, se define una función de pérdida que mide la discrepancia entre el score estimado por el modelo y el score real, y que se minimiza durante el proceso de entrenamiento.

Como $p_t(\mathbf{x})$ no se conoce de forma explícita, no se entrena directamente contra $\nabla_{\mathbf{x}} \log p_t(\mathbf{x})$. En su lugar, se utiliza la distribución de transición $p_{0t}(\mathbf{x}(t) \mid \mathbf{x}_0)$, que sí es conocida para los procesos gaussianos considerados.

La función de pérdida utilizada puede expresarse como:

$$\mathcal{L}(\theta) = \mathbb{E}_{t,\mathbf{x}_0,\mathbf{z}} \left[ \lambda(t) \left\| s_\theta(\mathbf{x}_t, t) + \frac{\mathbf{z}}{\sigma(t)} \right\|^2 \right] \tag{2.12}$$

donde $\mathbf{x}_t$ es la versión perturbada de la muestra original y $\lambda(t)$ es un peso dependiente del tiempo.

En los procesos gaussianos, si $\mathbf{x}_t = \mu_t(\mathbf{x}_0) + \sigma(t)\mathbf{z}$, entonces el score condicional viene dado por:

$$\nabla_{\mathbf{x}_t} \log p_{0t}(\mathbf{x}_t \mid \mathbf{x}_0) = -\frac{\mathbf{x}_t - \mu_t(\mathbf{x}_0)}{\sigma^2(t)} = -\frac{\mathbf{z}}{\sigma(t)}. \tag{2.13}$$

Por ello, entrenar la red para que $\sigma(t) s_\theta(\mathbf{x}_t, t)$ se aproxime a $-\mathbf{z}$ equivale a aprender el score de las distribuciones perturbadas.

Una vez aprendido este score, se puede utilizar para definir el proceso inverso de difusión. En particular, el proceso de generación puede implementarse mediante la integración numérica de una SDE inversa, utilizando esquemas como Euler–Maruyama o métodos predictor–corrector.

Este enfoque resulta especialmente potente porque evita la necesidad de modelar explícitamente la densidad de probabilidad, permitiendo trabajar en espacios de alta dimensión, como las imágenes.

## 2.4. Arquitectura U-Net

La arquitectura U-Net es uno de los diseños más utilizados en modelos de difusión y modelos basados en score, debido a su capacidad para capturar tanto detalles locales como estructuras globales en datos de alta dimensión, como imágenes.

U-Net presenta una arquitectura en forma de "U", compuesta por una ruta de compresión (*encoder*) y una ruta de expansión (*decoder*), conectadas mediante *skip connections*, que permiten combinar información de distintas resoluciones.

La ruta de compresión extrae representaciones cada vez más abstractas de la entrada mediante convoluciones y reducción de resolución, mientras que la ruta de expansión restaura progresivamente la resolución espacial. Las *skip connections* permiten recuperar detalles finos al combinar información de ambas rutas.

En el contexto de modelos de difusión, la red recibe como entrada una imagen perturbada $\mathbf{x}(t)$ junto con una representación del instante temporal $t$. Para incorporar esta información, se emplean embeddings basados en *Gaussian random Fourier features*, que permiten adaptar el modelo al nivel de ruido.

La salida de la red corresponde a una estimación del score $s_\theta(\mathbf{x}, t)$, que se utiliza para guiar el proceso inverso de difusión.

En este proyecto, se emplea una variante de U-Net con capas convolucionales, normalización por grupos (*Group Normalization*) y activaciones tipo *swish*, lo que permite un entrenamiento estable y buen rendimiento.

## 2.5. Métodos de muestreo

El proceso de generación de imágenes en modelos de difusión se realiza mediante un procedimiento de muestreo iterativo que aproxima la solución del proceso inverso de difusión. Dado que dicho proceso se describe mediante una ecuación diferencial estocástica dependiente del score, es necesario recurrir a métodos numéricos para generar muestras a partir de ruido inicial.

Sea la SDE inversa asociada al proceso de difusión:

$$\mathrm{d}\mathbf{x}(t) = \left[f(\mathbf{x}(t), t) - g(t)^2 \nabla_{\mathbf{x}} \log p_t(\mathbf{x}(t))\right] \mathrm{d}t + g(t)\mathrm{d}\mathbf{W}(t), \tag{2.14}$$

donde el término $\nabla_{\mathbf{x}} \log p_t(\mathbf{x}(t))$ es el score estimado por el modelo $s_\theta(\mathbf{x}, t)$. El objetivo del muestreo consiste en integrar numéricamente esta ecuación desde $t = T$ hasta $t = 0$ para obtener una muestra $\mathbf{x}_0$ distribuida según la distribución de los datos.

#### Método de Euler–Maruyama

El método de Euler–Maruyama es el integrador numérico más sencillo para ecuaciones diferenciales estocásticas y constituye la base de muchos esquemas de muestreo en modelos de difusión. Dado un paso temporal $\Delta t$, la actualización se expresa como:

$$\mathbf{x}_{t-\Delta t} = \mathbf{x}_t + \left[f(\mathbf{x}_t, t) - g(t)^2 s_\theta(\mathbf{x}_t, t)\right] \Delta t + g(t)\sqrt{\Delta t}\, \mathbf{z}, \tag{2.15}$$

donde $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ es ruido gaussiano isotrópico. Este método introduce estocasticidad explícita en cada paso y resulta sencillo de implementar, aunque puede requerir un número elevado de pasos para obtener muestras de alta calidad.

### 2.5.1. Métodos predictor–corrector

Los métodos predictor–corrector combinan un paso predictor, normalmente basado en Euler–Maruyama, con uno o varios pasos correctores diseñados para refinar la muestra.

$$\mathbf{x}^{(p)}_{t-\Delta t} = \mathbf{x}_t + \left[f(\mathbf{x}_t, t) - g(t)^2 s_\theta(\mathbf{x}_t, t)\right] \Delta t + g(t)\sqrt{\Delta t}\mathbf{z}. \tag{2.16}$$

El corrector suele implementarse mediante dinámica de Langevin, utilizando el score estimado para desplazar la muestra hacia regiones de mayor probabilidad.

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \epsilon\, s_\theta(\mathbf{x}^{(k)}, t) + \sqrt{2\epsilon}\mathbf{z}^{(k)}, \qquad \mathbf{z}^{(k)} \sim \mathcal{N}(0, \mathbf{I}). \tag{2.17}$$

Tras cada paso predictor, se aplican varios pasos correctores y la última muestra corregida se utiliza como punto de partida para el siguiente instante temporal. Este procedimiento combina la integración temporal con una corrección local basada en el score, que desplaza las muestras hacia regiones de mayor densidad, mejorando su calidad al reducir el error de discretización y estabilizar la trayectoria de muestreo, aunque a costa de un mayor costo computacional debido al número adicional de evaluaciones del modelo.

### 2.5.2. Probability Flow ODE

Es una alternativa determinista al muestreo estocástico, donde se considera una ecuación diferencial ordinaria cuya solución comparte las mismas distribuciones marginales que la SDE original. Esta formulación elimina el término de ruido, dando lugar a la ecuación:

$$\mathrm{d}\mathbf{x}(t) = \left[ f(\mathbf{x}(t), t) - \frac{1}{2} g(t)^2 \nabla_{\mathbf{x}} \log p_t(\mathbf{x}(t)) \right] \mathrm{d}t. \tag{2.18}$$

La diferencia principal respecto a la SDE inversa es el factor $\frac{1}{2}$ delante del término de score y la desaparición del término aleatorio. Aunque las trayectorias individuales son distintas, la familia de distribuciones marginales que se obtiene en cada instante coincide con la de la SDE.

La integración de esta ecuación permite generar muestras de forma determinista, lo que facilita el control del proceso y la evaluación del logaritmo de la verosimilitud. No obstante, el carácter determinista puede reducir la diversidad de las muestras generadas en comparación con los métodos estocásticos.

En términos comparativos, el método de Euler–Maruyama es sencillo y eficiente, pero puede requerir un gran número de pasos para obtener muestras de alta calidad. Los métodos predictor–corrector mejoran la calidad a costa de un mayor costo computacional, mientras que la probability flow ODE ofrece un enfoque determinista que permite, además, el cálculo de verosimilitudes; sin embargo, puede reducir la diversidad de las muestras generadas.

## 2.6. Métricas de evaluación

Para evaluar la calidad de las imágenes generadas, se emplean métricas estándar en modelos generativos.

### 2.6.1. Fréchet Inception Distance (FID)

El Fréchet Inception Distance (FID) mide la distancia entre las distribuciones de características de imágenes reales y generadas. Para ello, ambas distribuciones se aproximan mediante gaussianas multivariantes en el espacio de características de una red Inception preentrenada.

Sea $(\mu_r, \Sigma_r)$ la media y covarianza de las características de las imágenes reales, y $(\mu_g, \Sigma_g)$ las correspondientes a las imágenes generadas. El FID se define como:

$$\mathrm{FID} = \|\mu_r - \mu_g\|^2 + \mathrm{Tr}\left(\Sigma_r + \Sigma_g - 2(\Sigma_r \Sigma_g)^{1/2}\right). \tag{2.19}$$

Esta métrica evalúa simultáneamente la calidad y la diversidad de las muestras generadas. Valores más bajos indican una mayor similitud entre las distribuciones.

Una de sus principales ventajas es que tiene en cuenta correlaciones entre características, aunque depende de la red utilizada para la extracción de dichas características.

### 2.6.2. Inception Score (IS)

El Inception Score (IS) evalúa la calidad y diversidad de las imágenes generadas utilizando un modelo de clasificación preentrenado. La idea es que una imagen de alta calidad debe producir una distribución de clases con baja entropía, mientras que un conjunto diverso de imágenes debe tener una distribución marginal amplia.

Se define como:

$$\mathrm{IS} = \exp\left(\mathbb{E}_{\mathbf{x}}\left[D_{KL}\left(p(y|\mathbf{x}) \,\|\, p(y)\right)\right]\right), \tag{2.20}$$

donde $p(y|\mathbf{x})$ es la distribución de clases predicha por el modelo y $p(y)$ es la distribución marginal.

Valores más altos indican mejor calidad y mayor diversidad. Sin embargo, esta métrica no compara directamente con datos reales, por lo que puede no detectar ciertos tipos de artefactos.

### 2.6.3. Bits Per Dimension (BPD)

La métrica Bits Per Dimension (BPD) evalúa la calidad de un modelo generativo desde un punto de vista probabilístico, midiendo cuántos bits son necesarios, en promedio, para codificar cada dimensión de los datos bajo el modelo aprendido.

Se define como:

$$\mathrm{BPD} = -\frac{1}{D}\log_2 p(\mathbf{x}), \tag{2.21}$$

donde $D$ es el número total de dimensiones de la muestra.

Intuitivamente, un modelo con menor BPD es capaz de describir los datos utilizando menos información, lo que implica una mejor aproximación de la distribución subyacente.

En el caso de modelos de difusión continuos, la BPD se estima mediante la integración de la *probability flow ODE*, que permite calcular la densidad de probabilidad de muestras reales a partir de una transformación determinista hacia una distribución conocida.

Dado que las imágenes originales son discretas, se aplica *dequantización uniforme* antes del cálculo de la métrica. Este procedimiento introduce un pequeño ruido uniforme para tratar los datos como variables continuas, permitiendo una estimación coherente de la densidad bajo el modelo.

# Capítulo 3. Desarrollo de software

## 3.1. Análisis de Requisitos

### 3.1.1. Requisitos funcionales

- **RF-01. Carga y preprocesado de datos.** El sistema debe permitir cargar datasets de imágenes en escala de grises y en color, aplicando redimensionado, normalización y conversión a tensores.
- **RF-02. Configuración del proceso de difusión.** El sistema debe permitir seleccionar entre modelos VE, basados en movimiento browniano, y VP, basados en Ornstein–Uhlenbeck.
- **RF-03. Configuración del esquema de ruido.** El sistema debe permitir utilizar distintos schedules de ruido, incluyendo linear, cosine y sigmoid.
- **RF-04. Entrenamiento del modelo de score.** El sistema debe permitir entrenar una red U-Net mediante *denoising score matching* y almacenar checkpoints.
- **RF-05. Generación de imágenes sintéticas.** El sistema debe permitir generar imágenes mediante Euler–Maruyama, predictor–corrector y *probability flow ODE*.
- **RF-06. Evaluación de calidad.** El sistema debe permitir evaluar las imágenes generadas mediante FID, IS y BPD.
- **RF-07. Comparación experimental.** El sistema debe permitir comparar modelos, samplers y schedules mediante métricas y visualización.
- **RF-08. Generación controlable.** El sistema debe permitir generación condicionada por clase y la imputación de regiones.
- **RF-09. Configuración del usuario.** El sistema debe permitir seleccionar dataset, número de imágenes, modelo, sampler y parámetros.

### 3.1.2. Requisitos no funcionales

- **RNF-01. Modularidad.** El código debe organizarse en módulos independientes.
- **RNF-02. Reproducibilidad.** Se deben poder fijar semillas y reutilizar checkpoints.
- **RNF-03. Compatibilidad hardware.** El sistema debe ejecutarse en CPU, CUDA

o MPS.

- **RNF-04. Escalabilidad.** Debe soportar imágenes en escala de grises y RGB.
- **RNF-05. Rendimiento.** Los experimentos base deben ejecutarse con recursos académicos limitados.
- **RNF-06. Mantenibilidad.** Debe ser posible añadir nuevos módulos sin modificar la arquitectura.
- **RNF-07. Claridad.** El código y notebooks deben ser legibles.
- **RNF-08. Trazabilidad.** Los resultados deben identificarse por configuración experimental.

## 3.2. Casos de Uso

- **CU-01. Entrenamiento.** El usuario configura el experimento y entrena el modelo, generando un checkpoint.
- **CU-02. Generación.** El usuario carga un checkpoint y genera imágenes.
- **CU-03. Comparación.** Se comparan configuraciones mediante métricas y visualización.
- **CU-04. Generación condicionada.** El sistema genera imágenes condicionadas.
- **CU-05. Imputación.** El sistema reconstruye regiones faltantes.
- **CU-06. Visualización.** Se muestran resultados y evolución del proceso.

## 3.3. Arquitectura del sistema

El sistema se ha diseñado siguiendo una arquitectura modular, separando los distintos componentes del modelo de difusión en módulos independientes. Esta organización facilita la reutilización del código, la comparación entre configuraciones y la extensión del sistema.

La arquitectura se organiza en cinco capas:

- **Datos.** Carga y preprocesado de imágenes, incluyendo normalización y creación de *DataLoaders*.
- **Procesos de difusión.** Definición de los modelos VE y VP mediante sus funciones de deriva, difusión y distribuciones marginales.
- **Modelo neuronal.** Red de score basada en U-Net con embeddings temporales mediante *Gaussian random Fourier features*.

- **Generación.** Implementación de los samplers (Euler–Maruyama, predictor–corrector y *probability flow ODE*) y extensiones para conditioning e imputación.
- **Evaluación.** Cálculo de métricas (FID, IS, BPD) y visualización de resultados.

La lógica del sistema se implementa en módulos Python dentro de `src/`, mientras que los notebooks gestionan la configuración y ejecución de los experimentos.

Estructura del proyecto:

- `src/`: módulos principales.
- `train/`: entrenamiento.
- `samplers/`: generación.
- `measures/`: evaluación.
- `checkpoints/`: modelos.
- `samples/`: resultados.
- `data/`: datasets.

Los módulos principales incluyen la definición de los procesos de difusión, la red de score, los samplers, utilidades de visualización y métricas.

La arquitectura completa del sistema se resume en el diagrama de clases de la Figura 3.1
.

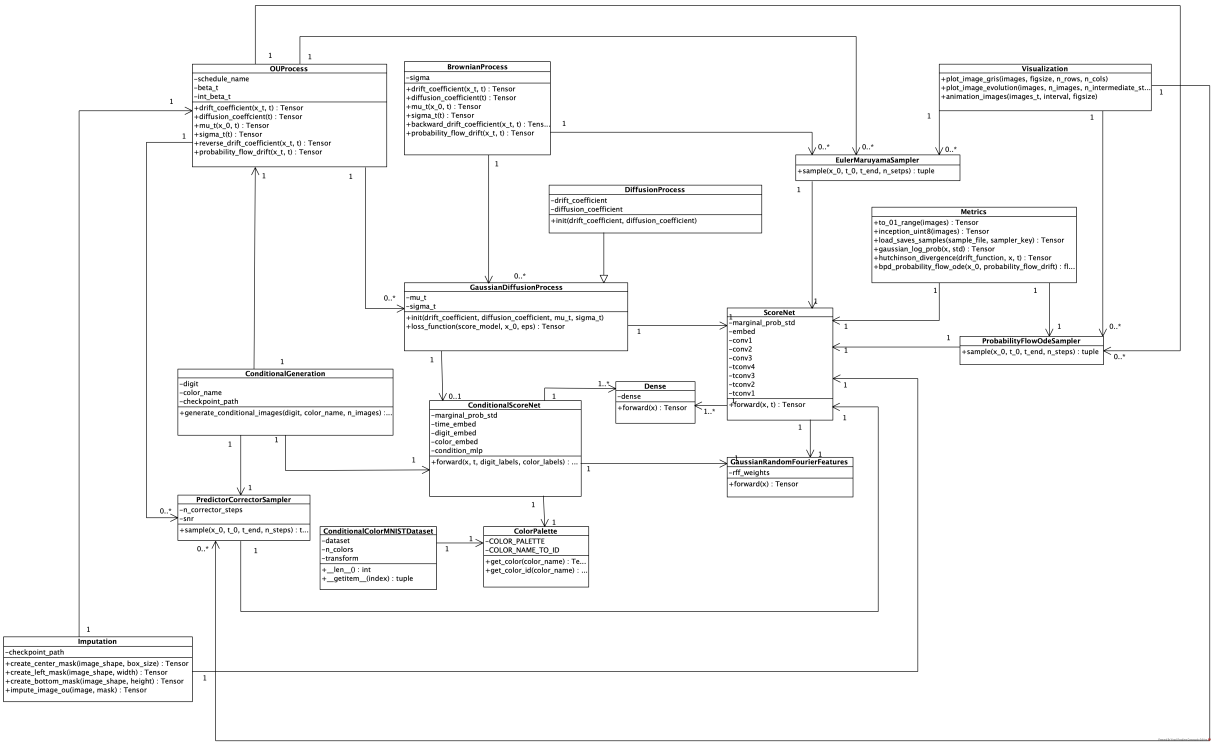



Figura 3.1: Diagrama de clases del sistema.

## 3.4. Validación y calidad del software

La validación del sistema combina pruebas funcionales, evaluación cuantitativa y análisis cualitativo.

- **Pruebas funcionales:** Verificación del proceso forward, la función de pérdida y los samplers.
- **Pruebas de integración:** Validación del pipeline completo.
- **Evaluación cuantitativa:** Uso de FID, IS y BPD. La BPD se calcula mediante estimación de log-verosimilitud con la *probability flow ODE*.
- **Evaluación cualitativa:** Inspección visual de las imágenes y del proceso de denoising.

Se comprobó que la generación evoluciona correctamente desde ruido hasta muestras estructuradas.

La reproducibilidad se garantiza mediante semillas y almacenamiento de resultados en formato `.pt`.

## 3.5. Planificación del proyecto

La planificación temporal completa se muestra en el diagrama de Gantt descrito en la Figura 3.2.

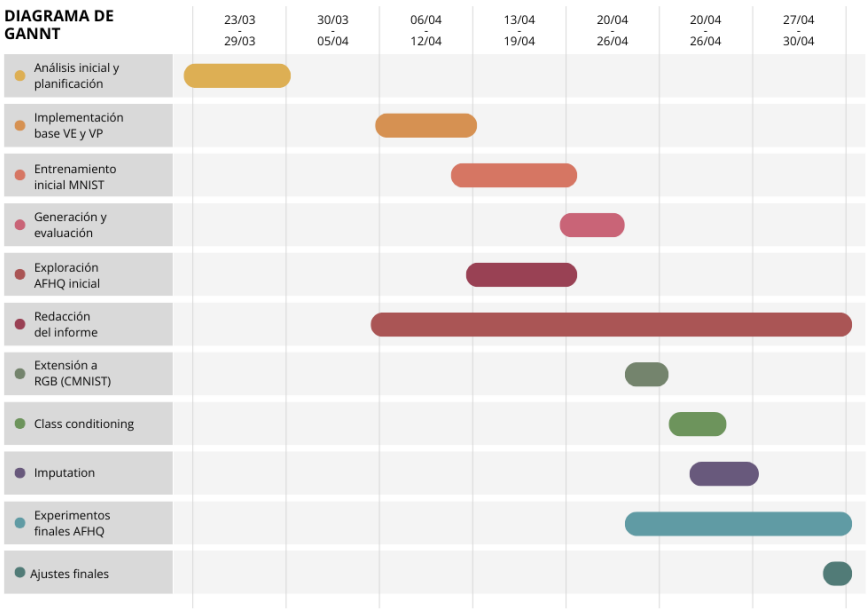



Figura 3.2: Diagrama de Gantt

# Capítulo 4. Experimentación y resultados

## 4.1. Configuración experimental

Los experimentos se realizaron principalmente sobre MNIST restringido al dígito 3, utilizado como entorno experimental controlado para comparar los distintos métodos de generación. Además, se consideró una versión de MNIST, denominada CMNIST, en la que las imágenes se transformaron a tres canales RGB mediante una coloración aleatoria.

De forma adicional, se exploró el dataset AFHQ de gatos como prueba de escalabilidad a imágenes RGB naturales. Dado que este experimento no constituye el eje principal del documento, sus resultados se presentan en el Apéndice III .

Las imágenes del dataset MNIST, originalmente en el rango discreto $[0, 255]$, se transformaron a valores continuos en el intervalo $[-1, 1]$ mediante normalización. Esta transformación es habitual en modelos de difusión, ya que mejora la estabilidad numérica y el comportamiento del modelo durante el entrenamiento.

Para la evaluación cuantitativa, se generaron 2000 imágenes por configuración, lo que permitió obtener estimaciones más estables sin que el coste computacional resultara excesivo. La evaluación se realizó sobre checkpoints correspondientes a 10, 20, 50, 100, 200 y 500 épocas de entrenamiento para los samplers de Euler–Maruyama, predictor–corrector y probability flow ODE. Además, el número de pasos de muestreo se fijó en 500, como compromiso entre precisión en la integración numérica y tiempo de generación.

El *batch size* utilizado durante el entrenamiento fue de 32, mientras que durante la generación se utilizó un *batch size* de 128, ya que este último únicamente determina cuántas imágenes se procesan simultáneamente durante el muestreo. Su elección respondió a criterios de eficiencia computacional y memoria disponible.

Las imágenes generadas se almacenaron en formato `.pt`, preservando directamente los tensores de PyTorch. Esto permitió reutilizar exactamente las mismas muestras para el cálculo posterior de métricas, evitando pérdidas de precisión o cambios de escala.

Los experimentos se ejecutaron en distintos entornos hardware, incluyendo GPU NVIDIA T4, A100 y H100 en Google Colab, así como la aceleración MPS en un MacBook. La disponibilidad de GPU condicionó directamente el tamaño del batch, el número de imágenes generadas y la duración de los entrenamientos.

Aunque se calcularon FID, IS y BPD para las configuraciones evaluadas, el FID se utilizó como métrica cuantitativa principal, ya que compara directamente las distribuciones de imágenes reales y generadas en un espacio de características.

En cuanto a la métrica BPD, se calculó mediante la integración de la *probability flow ODE*, lo que permite estimar la log-verosimilitud de muestras reales bajo el modelo generativo.

Para hacer la métrica comparable con la definición estándar de BPD en la literatura, se aplica una corrección de cambio de variable que tiene en cuenta tanto la dequantización uniforme como la transformación de escala (se añade un término constante de 7 bits por dimensión al valor estimado).

La integración se realizó mediante un esquema de Euler y la divergencia del campo de deriva se aproximó mediante el estimador de Hutchinson, por lo que los valores obtenidos deben interpretarse como estimaciones numéricas de la log-verosimilitud.

## 4.2. Evaluación del modelo VE

La Figura 4.1 muestra ejemplos representativos de la evolución del proceso de generación para los tres samplers. En todos los casos, se observa una transición desde ruido gaussiano hacia estructuras reconocibles del dígito 3.



(a) Euler–Maruyama (b) Predictor–Corrector (c) Probability Flow ODE

Figura 4.1: Comparación de los samplers para el modelo VE.

La Tabla 4.1 resume las configuraciones más relevantes, ordenadas según FID. El mejor valor de FID se obtuvo con la *probability flow ODE* en el checkpoint de 20 épocas, con un valor de 100.12. Sin embargo, la mejor BPD se alcanzó en checkpoints más avanzados, en torno a 500 épocas, con valores cercanos a 2.69. Este comportamiento indica que la mejora en la calidad probabilística del modelo se produce de forma más progresiva que la mejora perceptual medida por FID, cuyos mejores valores aparecen en etapas más tempranas del entrenamiento.

| **Sampler** | **Época** | **FID** | **IS** | **BPD** |
|---|---|---|---|---|
| ODE | 20 | 100.12 | 1.80 | 3.22 |
| PC | 50 | 102.74 | 1.51 | 2.79 |
| EM | 100 | 106.61 | 1.70 | 2.74 |
| ODE | 200 | 108.04 | 1.60 | 2.79 |
| ODE | 500 | 119.13 | 1.45 | 2.69 |

Cuadro 4.1: Configuraciones más relevantes del modelo VE según FID, IS y BPD.

Los resultados muestran que la *probability flow ODE* obtiene valores de FID competitivos, especialmente en checkpoints tempranos. No obstante, la inspección visual de las

muestras indicó que predictor–corrector generaba dígitos más estables y reconocibles en varias configuraciones. Esta discrepancia sugiere que, en un dataset simple como MNIST, FID no siempre captura completamente la calidad perceptual de las muestras.

Por otro lado, la BPD mejora progresivamente a medida que avanza el entrenamiento, alcanzando sus mejores valores en etapas tardías. Este resultado sugiere que una mejor estimación de la densidad (menor BPD) no implica necesariamente una mejora equivalente en la calidad visual de las muestras. Por este motivo, no se seleccionó la ODE como el mejor sampler de forma automática, a pesar de sus buenos valores de FID, y se decidió extender el análisis al modelo VP prestando especial atención a los samplers ODE y predictor–corrector.

## 4.3. Evaluación del modelo VP

Tras la evaluación del modelo VE, se analizó el comportamiento del modelo VP basado en el proceso de Ornstein–Uhlenbeck. En este caso, se estudiaron tres *schedules* de ruido: *linear*, *cosine* y *sigmoid*. Dado que en la evaluación VE los samplers ODE y predictor–corrector fueron los más competitivos, el análisis se centró principalmente en estos dos métodos, incluyendo Euler–Maruyama como referencia.

La Figura II.4 muestra la evolución del FID para los distintos *schedules* y samplers, permitiendo analizar de forma conjunta el efecto del perfil de ruido y del método de muestreo.

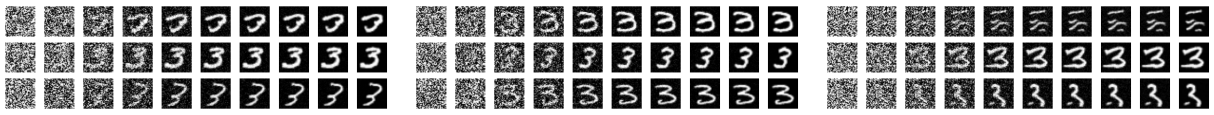
Los resultados cuantitativos indican que el *schedule cosine* proporciona el mejor rendimiento global. En particular, la configuración con sampler predictor–corrector alcanza el menor valor de FID observado en los experimentos, con un valor de 18.23 en el checkpoint de 50 épocas. Este resultado es significativamente mejor que los obtenidos con los *schedules* *linear* y *sigmoid*, cuyos mejores valores se sitúan en torno a 30.03 y 39.34 respectivamente.

La Tabla 4.2 resume las mejores configuraciones obtenidas para cada *schedule*, incluyendo FID, IS y BPD.

| **Schedule** | **Sampler** | **Época** | **FID** | **IS** | **BPD** |
|---|---|---|---|---|---|
| Linear | ODE | 200 | 30.03 | 1.60 | 1.48 |
| Cosine | PC | 50 | 18.23 | 1.61 | 1.70 |
| Sigmoid | ODE | 200 | 39.34 | 1.71 | 2.01 |

Cuadro 4.2: Mejores configuraciones del modelo VP según FID, IS y BPD.

El *schedule cosine* con predictor–corrector destaca frente al resto de configuraciones tanto en términos de FID como en calidad visual. La Figura 4.2 ilustra este comportamiento, mostrando muestras más nítidas y consistentes que las obtenidas con otros samplers.



(a) Euler–Maruyama (b) Predictor–corrector (c) Probability Flow ODE

Figura 4.2: Comparación de los samplers para el modelo VP con schedule cosine.

En cuanto a la métrica BPD, se observa un comportamiento coherente con la evolución del entrenamiento. Los valores disminuyen progresivamente a medida que el modelo mejora su aproximación a la distribución de los datos, alcanzando valores en torno a 1.0–1.5 bits por dimensión en los mejores checkpoints.

Sin embargo, estos mínimos de BPD no coinciden con los mejores valores de FID, que se alcanzan en etapas más tempranas del entrenamiento. Este desacoplamiento indica que una mejor estimación de la verosimilitud no implica necesariamente una mejora equivalente en la calidad perceptual de las muestras generadas.

De forma similar, la métrica IS presenta variaciones relativamente pequeñas entre configuraciones, lo que limita su capacidad para discriminar entre modelos en un dataset de complejidad moderada como MNIST.

## 4.4. Comparación final entre VE y VP

Una vez identificadas las mejores configuraciones dentro de cada familia de modelos, se realizó una comparación final entre el modelo VE y el modelo VP. Para VE se consideró la configuración basada en probability flow ODE, mientras que para VP se seleccionó la combinación de schedule *cosine* y sampler predictor–corrector.

La Figura II.7 muestra la evolución del FID para ambas configuraciones.

Por un lado, el modelo VP alcanza su mejor rendimiento en torno a 50 épocas, con un valor de FID de 18.23, mientras que el modelo VE presenta valores significativamente superiores en todas las configuraciones evaluadas, con un mínimo cercano a 100.12.

Además, se observa que el modelo VP converge más rápidamente, alcanzando buenos resultados en etapas tempranas del entrenamiento, mientras que el modelo VE muestra una mejora más lenta y limitada incluso en épocas avanzadas.

En cuanto a la métrica BPD, el modelo VP también presenta valores más bajos en las configuraciones más avanzadas, frente a valores superiores en VE. Esto indica que, además de generar muestras visualmente más realistas, el modelo VP proporciona una mejor estimación de la densidad de probabilidad de los datos.

Por otro lado, el Inception Score (IS) no muestra diferencias tan marcadas entre ambas formulaciones, manteniéndose en rangos similares.

En conjunto, los resultados muestran que el modelo VP no solo supera al modelo VE en términos de calidad visual (FID), sino también en términos probabilísticos (BPD), ofreciendo una dinámica de difusión más estable y eficiente. La presencia del término de deriva en el proceso de Ornstein–Uhlenbeck permite controlar la evolución de la varianza, facilitando el aprendizaje del modelo y mejorando el proceso de generación.

Por tanto, se selecciona como mejor configuración final el modelo VP con schedule *cosine* y sampler predictor–corrector, que será utilizado en las extensiones posteriores del sistema.

## 4.5. Generación condicional de imágenes a color

Con el objetivo de dotar al sistema de capacidad de generación controlable, se implementó un modelo de difusión condicionado simultáneamente en el dígito y en el color de las imágenes. Para ello, se construyó un dataset basado en MNIST en el que cada imagen se transforma a formato RGB utilizando una paleta discreta de colores predefinida.

Cada muestra del dataset está asociada a dos etiquetas: el dígito original y un identificador de color perteneciente a la paleta. De este modo, el modelo aprende una distribución condicional $p(\mathbf{x} \mid y_{\text{digit}}, y_{\text{color}})$.

Para incorporar esta información en el modelo, la arquitectura de la red de score se extendió mediante embeddings aprendidos tanto para el dígito como para el color, que se combinan con la codificación temporal del proceso de difusión. Esta representación conjunta permite que el modelo adapte la generación en función de las condiciones especificadas.

Durante la fase de generación, se fijan explícitamente el dígito y el color deseados, lo que permite sintetizar imágenes coherentes con ambas condiciones. El proceso de muestreo se realiza utilizando el modelo VP con schedule cosine y el sampler predictor–corrector, que previamente mostró el mejor comportamiento en los experimentos sobre MNIST.

La Figura 4.3 muestra ejemplos representativos de imágenes generadas.

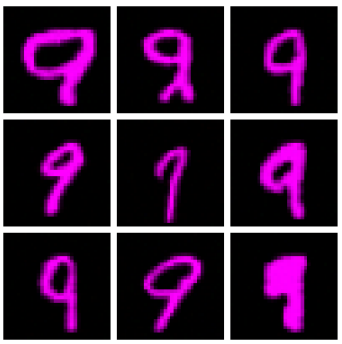


Figura 4.3: Ejemplo de generación condicional para el número 9 y color magenta.

Los resultados muestran que el modelo es capaz de controlar de forma efectiva ambas

variables, generando dígitos reconocibles y coherentes con el color seleccionado. Esto indica que la arquitectura propuesta es capaz de desacoplar la información estructural de los atributos adicionales, permitiendo una generación flexible y controlada.

Este experimento valida la capacidad del sistema para incorporar variables condicionantes en el proceso generativo, lo que constituye un paso previo fundamental para tareas más avanzadas como la imputación o la generación guiada por atributos.

## 4.6. Imputing

Además de la generación de imágenes desde ruido, se evaluó la capacidad del modelo para realizar la imputación de regiones. Esta tarea consiste en reconstruir una parte oculta de una imagen utilizando el modelo generativo y manteniendo fija la información observada.

Para ello, se parte de una imagen real $\mathbf{x}$ y de una máscara binaria $\mathbf{m}$, donde los píxeles con valor 1 corresponden a regiones conocidas y los píxeles con valor 0 a la región que se desea reconstruir. En el experimento realizado se ocultó una región cuadrada.

Durante el proceso inverso de difusión, la parte desconocida se genera mediante predictor–corrector, mientras que la parte conocida se reemplaza en cada paso por una versión ruidosa coherente con el instante temporal actual. De este modo, el modelo no solo genera contenido plausible, sino que además lo hace condicionado por los píxeles observados.

Formalmente, en cada instante $t$ se combina la muestra generada $\mathbf{x}_t$ con la imagen original perturbada $\tilde{\mathbf{x}}_t$ mediante la máscara:

$$\mathbf{x}_t \leftarrow (1 - \mathbf{m}) \odot \mathbf{x}_t + \mathbf{m} \odot \tilde{\mathbf{x}}_t.$$

Al finalizar el proceso, los píxeles conocidos se sustituyen por los valores originales, conservando únicamente la región reconstruida por el modelo.

La Figura 4.4 muestra un ejemplo del procedimiento. A la izquierda se presenta la imagen original, en el centro la imagen con una región oculta y a la derecha el resultado imputado. Se observa que el modelo es capaz de reconstruir una región compatible con la forma global del dígito, manteniendo la coherencia visual con las partes observadas.

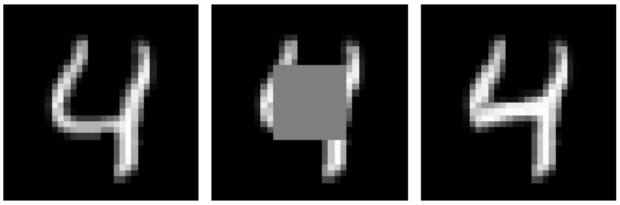


Figura 4.4: Imagen original, imagen con región oculta y reconstrucción generada.

# Capítulo 5. Conclusiones

Los experimentos realizados permiten extraer varias conclusiones relevantes acerca del comportamiento de los modelos de difusión y de los distintos esquemas de muestreo evaluados.

En primer lugar, el modelo VP basado en el proceso de Ornstein–Uhlenbeck mostró un comportamiento más estable y robusto que el modelo VE basado en movimiento browniano en el caso del dataset MNIST. En particular, la combinación del *schedule* cosine con el esquema predictor–corrector proporcionó el mejor rendimiento global, tanto en términos de métricas cuantitativas como de calidad visual de las muestras generadas.

En segundo lugar, se observó que las métricas cuantitativas no siempre reflejan completamente la calidad perceptual de las imágenes generadas. Aunque en algunos casos la *probability flow ODE* obtuvo valores competitivos de FID, el método predictor–corrector generó muestras visualmente más definidas y reconocibles. Este resultado pone de manifiesto la importancia de complementar las métricas objetivas con un análisis cualitativo, especialmente en tareas de generación de imágenes.

En tercer lugar, la comparación entre MNIST y CMNIST evidencia que la generación de imágenes en color introduce una mayor complejidad en el problema. Mientras que el modelo converge rápidamente en el caso de imágenes en escala de grises, la extensión al espacio RGB requiere un mayor número de épocas de entrenamiento para alcanzar resultados comparables. Este comportamiento es consistente con el incremento en la dimensionalidad y la estructura de dependencia entre canales.

Finalmente, los resultados obtenidos en el dataset AFHQ (véase Apéndice III) ponen de relieve las limitaciones del sistema en escenarios más complejos. La generación de imágenes naturales de alta variabilidad requiere arquitecturas más expresivas, conjuntos de datos de mayor tamaño y entrenamientos más prolongados. No obstante, el sistema desarrollado ha demostrado ser capaz de abordar de forma completa el pipeline de modelos de difusión, incluyendo carga de datos, entrenamiento, generación y evaluación, cumpliendo así los objetivos planteados en el proyecto.

En conjunto, este trabajo confirma la eficacia de los modelos de difusión como herramientas para la generación de imágenes, así como la relevancia de la elección del proceso de difusión, el *schedule* de ruido y el método de muestreo en el rendimiento final del sistema.

# Bibliografía


[1] Brian D. O. Anderson. Reverse-time diffusion equation models. *Stochastic Processes and their Applications*, 12(3):313–326, 1982.

[2] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. In *Advances in Neural Information Processing Systems*, 2020.

[3] Aapo Hyvärinen. Estimation of non-normalized statistical models by score matching. *Journal of Machine Learning Research*, 6:695–709, 2005.

[4] Alexander Quinn Nichol and Prafulla Dhariwal. Improved denoising diffusion probabilistic models. In *International Conference on Machine Learning*, pages 8162–8171, 2021.

[5] Jascha Sohl-Dickstein, Eric A. Weiss, Niru Maheswaranathan, and Surya Ganguli. Deep unsupervised learning using nonequilibrium thermodynamics. In *International Conference on Machine Learning*, pages 2256–2265, 2015.

[6] Yang Song and Stefano Ermon. Generative modeling by estimating gradients of the data distribution. In *Advances in Neural Information Processing Systems*, volume 32, 2019.

[7] Yang Song, Jascha Sohl-Dickstein, Diederik Kingma, Abhishek Kumar, Stefano Ermon, and Ben Poole. Score-based generative modeling through stochastic differential equations. *International Conference on Learning Representations*, 2021.

[8] Alberto Suárez. Stochastic differential equations. Course notes, Universidad Autónoma de Madrid, 2026. Material docente utilizado en la asignatura.

[9] Pascal Vincent. A connection between score matching and denoising autoencoders. *Neural Computation*, 23(7):1661–1674, 2011.

[10] Lilian Weng. What are diffusion models? `https://lilianweng.github.io/posts/2021-07-11-diffusion-models/`, 2021. Accessed: 2026-04-27.

# Apéndice I. Derivaciones teóricas

## I.1. Derivación de la ecuación de Fokker–Planck

Consideremos la ecuación diferencial estocástica de Itô

$$dX(t) = f(X(t), t)\,\mathrm{d}t + \sqrt{2D(X(t), t)}\,\mathrm{d}W(t),$$

donde $f(x, t)$ es el término de deriva, $D(x, t)$ es el coeficiente de difusión y $W(t)$ es un proceso de Wiener estándar.

El objetivo es obtener una ecuación para la evolución temporal de la densidad de probabilidad $p(x, t)$ asociada al proceso $X(t)$.

Sea $\varphi(x)$ una función de prueba suficientemente regular. Por definición de esperanza,

$$\mathbb{E}[\varphi(X(t))] = \int \varphi(x)p(x, t)\,dx.$$

Derivando respecto al tiempo,

$$\frac{\mathrm{d}}{\mathrm{d}t}\mathbb{E}[\varphi(X(t))] = \frac{\mathrm{d}}{\mathrm{d}t}\int \varphi(x)p(x, t)\,dx.$$

Como $\varphi(x)$ no depende explícitamente de $t$, se obtiene

$$\frac{\mathrm{d}}{\mathrm{d}t}\mathbb{E}[\varphi(X(t))] = \int \varphi(x)\frac{\partial p(x, t)}{\partial t}\,dx.$$

Por otro lado, aplicando la fórmula de Itô a $\varphi(X(t))$,

$$d\varphi(X(t)) = \varphi'(X(t))\,dX(t) + \frac{1}{2}\varphi''(X(t))(dX(t))^2.$$

Sustituyendo la SDE,

$$dX(t) = f(X(t), t)\,\mathrm{d}t + \sqrt{2D(X(t), t)}\,\mathrm{d}W(t).$$

Usando las reglas de cálculo de Itô,

$$(\mathrm{d}t)^2 = 0, \qquad \mathrm{d}t\,\mathrm{d}W(t) = 0, \qquad (\mathrm{d}W(t))^2 = \mathrm{d}t,$$

se obtiene

$$(dX(t))^2 = 2D(X(t),t)\,\mathrm{d}t.$$

Por tanto,

$$d\varphi(X(t)) = \varphi'(X(t))f(X(t),t)\,\mathrm{d}t + \varphi'(X(t))\sqrt{2D(X(t),t)}\,\mathrm{d}W(t) + D(X(t),t)\varphi''(X(t))\,\mathrm{d}t.$$

Agrupando los términos en $\mathrm{d}t$ y $\mathrm{d}W(t)$,

$$d\varphi(X(t)) = \left[f(X(t),t)\varphi'(X(t)) + D(X(t),t)\varphi''(X(t))\right]\mathrm{d}t + \sqrt{2D(X(t),t)}\varphi'(X(t))\,\mathrm{d}W(t).$$

Tomando esperanza en ambos lados, el término estocástico desaparece porque tiene esperanza nula:

$$\mathbb{E}\left[\sqrt{2D(X(t),t)}\varphi'(X(t))\,\mathrm{d}W(t)\right] = 0.$$

Así,

$$\frac{\mathrm{d}}{\mathrm{d}t}\mathbb{E}[\varphi(X(t))] = \mathbb{E}\left[f(X(t),t)\varphi'(X(t)) + D(X(t),t)\varphi''(X(t))\right].$$

Escribiendo esta igualdad en términos de la densidad $p(x,t)$,

$$\int \varphi(x)\frac{\partial p(x,t)}{\partial t}\,dx = \int \left[f(x,t)\varphi'(x) + D(x,t)\varphi''(x)\right]p(x,t)\,dx.$$

Ahora integramos por partes el primer término del lado derecho:

$$\int f(x,t)p(x,t)\varphi'(x)\,dx = -\int \varphi(x)\frac{\partial}{\partial x}\left[f(x,t)p(x,t)\right]dx,$$

asumiendo que los términos de frontera se anulan.

Para el segundo término, se integra por partes dos veces:

$$\int D(x,t)p(x,t)\varphi''(x)\,dx = \int \varphi(x)\frac{\partial^2}{\partial x^2}\left[D(x,t)p(x,t)\right]dx.$$

Sustituyendo ambas expresiones,

$$\int \varphi(x)\frac{\partial p(x,t)}{\partial t}\,dx = \int \varphi(x)\left[-\frac{\partial}{\partial x}\left(f(x,t)p(x,t)\right) + \frac{\partial^2}{\partial x^2}\left(D(x,t)p(x,t)\right)\right]dx.$$

Como esta igualdad debe cumplirse para toda función de prueba $\varphi(x)$, los integrandos deben coincidir. Por tanto,

$$\frac{\partial p(x,t)}{\partial t} = -\frac{\partial}{\partial x}\left(f(x,t)p(x,t)\right) + \frac{\partial^2}{\partial x^2}\left(D(x,t)p(x,t)\right).$$

Esta es la ecuación de Fokker–Planck asociada a la SDE inicial.

En el caso multidimensional, para una SDE de la forma

$$d\mathbf{X}(t) = \mathbf{f}(\mathbf{X}(t),t)\,\mathrm{d}t + \mathbf{G}(\mathbf{X}(t),t)\,\mathrm{d}\mathbf{W}(t),$$

la ecuación de Fokker–Planck se escribe como

$$\frac{\partial p(\mathbf{x},t)}{\partial t} = -\sum_i \frac{\partial}{\partial x_i}\left[f_i(\mathbf{x},t)p(\mathbf{x},t)\right] + \frac{1}{2}\sum_{i,j}\frac{\partial^2}{\partial x_i \partial x_j}\left[D_{ij}(\mathbf{x},t)p(\mathbf{x},t)\right],$$

donde

$$\mathbf{D}(\mathbf{x},t) = \mathbf{G}(\mathbf{x},t)\mathbf{G}(\mathbf{x},t)^\top.$$

Esta ecuación describe la evolución temporal de la densidad de probabilidad del proceso, mientras que la SDE describe la evolución de sus trayectorias individuales.

## I.2. Solución marginal del proceso Browniano VE

Consideremos el proceso de difusión VE basado en movimiento browniano con coeficiente de difusión dependiente del tiempo:

$$d\mathbf{x}(t) = g(t)\,\mathrm{d}\mathbf{W}(t),$$

con condición inicial

$$\mathbf{x}(0) = \mathbf{x}_0.$$

En el caso considerado en este trabajo,

$$g(t) = \sigma^t, \qquad \sigma > 0.$$

El objetivo es obtener la distribución condicional de $\mathbf{x}(t)$ dado $\mathbf{x}(0) = \mathbf{x}_0$.

Integrando la SDE entre 0 y $t$, se obtiene

$$\mathbf{x}(t) = \mathbf{x}_0 + \int_0^t g(s)\,\mathrm{d}\mathbf{W}(s).$$

Como $g(s)$ es una función determinista del tiempo, la integral de Itô

$$\int_0^t g(s)\,\mathrm{d}\mathbf{W}(s)$$

es una variable aleatoria gaussiana de media cero. Por tanto,

$$\mathbb{E}\left[\int_0^t g(s)\,\mathrm{d}\mathbf{W}(s)\right] = \mathbf{0}.$$

De este modo, la media condicional del proceso es

$$\mathbb{E}\left[\mathbf{x}(t) \mid \mathbf{x}(0) = \mathbf{x}_0\right] = \mathbf{x}_0.$$

Para calcular la covarianza, usamos la isometría de Itô. Como el ruido es isotrópico,

$$\mathrm{Cov}\left[\int_0^t g(s)\,\mathrm{d}\mathbf{W}(s)\right] = \left(\int_0^t g^2(s)\,ds\right)\mathbf{I}.$$

Por tanto,

$$\mathrm{Cov}\left[\mathbf{x}(t) \mid \mathbf{x}(0) = \mathbf{x}_0\right] = \left(\int_0^t g^2(s)\,ds\right)\mathbf{I}.$$

Sustituyendo $g(s) = \sigma^s$,

$$\int_0^t g^2(s)\,ds = \int_0^t \sigma^{2s}\,ds.$$

Para resolver la integral, escribimos

$$\sigma^{2s} = e^{2s\log\sigma}.$$

Entonces,

$$\int_0^t \sigma^{2s}\,ds = \int_0^t e^{2s\log\sigma}\,ds = \left[\frac{e^{2s\log\sigma}}{2\log\sigma}\right]_0^t.$$

Evaluando en los extremos,

$$\int_0^t \sigma^{2s}\,ds = \frac{e^{2t\log\sigma} - 1}{2\log\sigma}.$$

Como

$$e^{2t\log\sigma} = \sigma^{2t},$$

se obtiene

$$\int_0^t \sigma^{2s}\, ds = \frac{\sigma^{2t}-1}{2\log\sigma}.$$

Definimos entonces

$$\sigma_t^2 = \frac{\sigma^{2t}-1}{2\log\sigma}.$$

Por tanto, la solución marginal condicionada a la muestra inicial es

$$\mathbf{x}(t) \mid \mathbf{x}(0) = \mathbf{x}_0 \sim \mathcal{N}\left(\mathbf{x}_0, \sigma_t^2\mathbf{I}\right),$$

es decir,

$$\mathbf{x}(t) \mid \mathbf{x}(0) = \mathbf{x}_0 \sim \mathcal{N}\left(\mathbf{x}_0, \frac{\sigma^{2t}-1}{2\log\sigma}\mathbf{I}\right).$$

Esta expresión muestra que la media del proceso permanece centrada en la muestra original, mientras que la varianza aumenta con el tiempo. Por esta razón, este proceso se denomina *Variance Exploding*.

## I.3. Solución marginal del proceso Ornstein–Uhlenbeck VP

Consideremos el proceso de difusión VP basado en el proceso de Ornstein–Uhlenbeck con coeficientes dependientes del tiempo:

$$d\mathbf{x}(t) = -\frac{1}{2}\beta(t)\mathbf{x}(t)\,\mathrm{d}t + \sqrt{\beta(t)}\,\mathrm{d}\mathbf{W}(t),$$

con condición inicial

$$\mathbf{x}(0) = \mathbf{x}_0.$$

El objetivo es obtener la distribución condicional de $\mathbf{x}(t)$ dado $\mathbf{x}(0) = \mathbf{x}_0$.

Se trata de una SDE lineal. Para resolverla, definimos el factor integrante

$$M(t) = \exp\left(\frac{1}{2}\int_0^t \beta(s)\, ds\right).$$

Multiplicamos el proceso por este factor e introducimos la variable auxiliar

$$\mathbf{y}(t) = M(t)\mathbf{x}(t).$$

Aplicando la regla del producto de Itô,

$$d\mathbf{y}(t) = M(t)d\mathbf{x}(t) + \mathbf{x}(t)dM(t) + dM(t)d\mathbf{x}(t).$$

Como $M(t)$ es determinista, se tiene

$$dM(t) = \frac{1}{2}\beta(t)M(t)\,\mathrm{d}t.$$

Además,

$$dM(t)d\mathbf{x}(t) = 0,$$

ya que $dM(t)$ solo contiene términos en $\mathrm{d}t$.

Sustituyendo la SDE original,

$$d\mathbf{y}(t) = M(t)\left[-\frac{1}{2}\beta(t)\mathbf{x}(t)\,\mathrm{d}t + \sqrt{\beta(t)}\,\mathrm{d}\mathbf{W}(t)\right] + \mathbf{x}(t)\left[\frac{1}{2}\beta(t)M(t)\,\mathrm{d}t\right].$$

Los términos de deriva se cancelan:

$$-\frac{1}{2}\beta(t)M(t)\mathbf{x}(t)\,\mathrm{d}t + \frac{1}{2}\beta(t)M(t)\mathbf{x}(t)\,\mathrm{d}t = 0.$$

Por tanto,

$$d\mathbf{y}(t) = M(t)\sqrt{\beta(t)}\,\mathrm{d}\mathbf{W}(t).$$

Integrando entre 0 y $t$,

$$\mathbf{y}(t) - \mathbf{y}(0) = \int_0^t M(s)\sqrt{\beta(s)}\,\mathrm{d}\mathbf{W}(s).$$

Como $M(0) = 1$ y $\mathbf{y}(0) = \mathbf{x}_0$, se obtiene

$$M(t)\mathbf{x}(t) = \mathbf{x}_0 + \int_0^t M(s)\sqrt{\beta(s)}\,\mathrm{d}\mathbf{W}(s).$$

Multiplicando por $M(t)^{-1}$,

$$\mathbf{x}(t) = M(t)^{-1}\mathbf{x}_0 + \int_0^t M(t)^{-1}M(s)\sqrt{\beta(s)}\,\mathrm{d}\mathbf{W}(s).$$

Como

$$M(t)^{-1} = \exp\left(-\frac{1}{2}\int_0^t \beta(u)\,du\right),$$

y

$$M(t)^{-1}M(s) = \exp\left(-\frac{1}{2}\int_s^t \beta(u)\,du\right),$$

la solución puede escribirse como

$$\mathbf{x}(t) = \mathbf{x}_0 \exp\left(-\frac{1}{2}\int_0^t \beta(u)\,du\right) + \int_0^t \exp\left(-\frac{1}{2}\int_s^t \beta(u)\,du\right)\sqrt{\beta(s)}\,\mathrm{d}\mathbf{W}(s).$$

El primer término es determinista y el segundo es una integral de Itô de una función determinista. Por tanto, $\mathbf{x}(t) \mid \mathbf{x}(0) = \mathbf{x}_0$ es una variable gaussiana.

La media condicional es

$$\mathbb{E}\left[\mathbf{x}(t) \mid \mathbf{x}(0) = \mathbf{x}_0\right] = \mathbf{x}_0 \exp\left(-\frac{1}{2}\int_0^t \beta(u)\,du\right).$$

Para calcular la covarianza, aplicamos la isometría de Itô:

$$\mathrm{Cov}\left[\mathbf{x}(t) \mid \mathbf{x}(0) = \mathbf{x}_0\right] = \left[\int_0^t \exp\left(-\int_s^t \beta(u)\,du\right)\beta(s)\,ds\right]\mathbf{I}.$$

Ahora calculamos la integral

$$\int_0^t \exp\left(-\int_s^t \beta(u)\,du\right)\beta(s)\,ds.$$

Observamos que

$$\frac{\mathrm{d}}{ds}\exp\left(-\int_s^t \beta(u)\,du\right) = \beta(s)\exp\left(-\int_s^t \beta(u)\,du\right).$$

Por tanto,

$$\int_0^t \exp\left(-\int_s^t \beta(u)\,du\right)\beta(s)\,ds = \left[\exp\left(-\int_s^t \beta(u)\,du\right)\right]_{s=0}^{s=t}.$$

Evaluando en los extremos,

$$\left[\exp\left(-\int_s^t \beta(u)\,du\right)\right]_{s=0}^{s=t} = 1 - \exp\left(-\int_0^t \beta(u)\,du\right).$$

Definimos entonces

$$\alpha(t) = \exp\left(-\frac{1}{2}\int_0^t \beta(u)\,du\right),$$

y

$$\sigma^2(t) = 1 - \exp\left(-\int_0^t \beta(u)\,du\right).$$

Así, la distribución marginal condicionada a la muestra inicial es

$$\mathbf{x}(t) \mid \mathbf{x}(0) = \mathbf{x}_0 \sim \mathcal{N}\left(\alpha(t)\mathbf{x}_0, \sigma^2(t)\mathbf{I}\right),$$

es decir,

$$\mathbf{x}(t) \mid \mathbf{x}(0) = \mathbf{x}_0 \sim \mathcal{N}\left(\mathbf{x}_0 \exp\left(-\frac{1}{2}\int_0^t \beta(u)\,du\right), \left[1 - \exp\left(-\int_0^t \beta(u)\,du\right)\right]\mathbf{I}\right).$$

Esta expresión muestra que la media decae progresivamente hacia cero, mientras que la varianza permanece acotada. Por esta razón, este proceso se denomina *Variance Preserving*.

## I.4. Derivación de la pérdida de denoising score matching

El entrenamiento del modelo se basa en aproximar la función de score dependiente del tiempo,

$$s_\theta(\mathbf{x}, t) \approx \nabla_{\mathbf{x}} \log p_t(\mathbf{x}).$$

Para ello se parte del objetivo de score matching ponderado:

$$\theta^* = \arg\min_\theta \mathbb{E}_{t\sim U[0,1]}\left[\lambda(t)\mathbb{E}_{\mathbf{x}\sim p_t}\left[\|s_\theta(\mathbf{x}, t) - \nabla_{\mathbf{x}} \log p_t(\mathbf{x})\|_2^2\right]\right].$$

Sin embargo, la densidad marginal $p_t(\mathbf{x})$ no se conoce explícitamente, por lo que tampoco se conoce su score. Para evitar este problema, se utiliza el score de la distribución condicional de perturbación $p_{t|0}(\mathbf{x}(t) \mid \mathbf{x}_0)$, que sí es conocida en los procesos gaussianos considerados.

Para un proceso de difusión gaussiano, se tiene

$$\mathbf{x}(t) = \mu_{\mathbf{x}_0}(t) + \sigma(t)\mathbf{Z}, \qquad \mathbf{Z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}).$$

Por tanto,

$$\mathbf{x}(t) \mid \mathbf{x}_0 \sim \mathcal{N}\left(\mu_{\mathbf{x}_0}(t), \sigma^2(t)\mathbf{I}\right).$$

La densidad condicional es

$$p_{t|0}(\mathbf{x}(t) \mid \mathbf{x}_0) = \frac{1}{(2\pi)^{D/2}\sigma^D(t)} \exp\left(-\frac{1}{2\sigma^2(t)} \left\|\mathbf{x}(t) - \mu_{\mathbf{x}_0}(t)\right\|_2^2\right).$$

Tomando logaritmos,

$$\log p_{t|0}(\mathbf{x}(t) \mid \mathbf{x}_0) = -\frac{D}{2}\log(2\pi) - D\log\sigma(t) - \frac{1}{2\sigma^2(t)} \left\|\mathbf{x}(t) - \mu_{\mathbf{x}_0}(t)\right\|_2^2.$$

Derivando respecto a $\mathbf{x}(t)$,

$$\nabla_{\mathbf{x}} \log p_{t|0}(\mathbf{x}(t) \mid \mathbf{x}_0) = -\frac{1}{\sigma^2(t)} \left(\mathbf{x}(t) - \mu_{\mathbf{x}_0}(t)\right).$$

Como

$$\mathbf{x}(t) - \mu_{\mathbf{x}_0}(t) = \sigma(t)\mathbf{Z},$$

se obtiene

$$\nabla_{\mathbf{x}} \log p_{t|0}(\mathbf{x}(t) \mid \mathbf{x}_0) = -\frac{\mathbf{Z}}{\sigma(t)}.$$

Sustituyendo este score condicional en el objetivo de denoising score matching,

$$\mathcal{L}(\theta) = \mathbb{E}_{t,\mathbf{x}_0,\mathbf{Z}}\left[\lambda(t) \left\| s_\theta(\mathbf{x}(t), t) + \frac{\mathbf{Z}}{\sigma(t)} \right\|_2^2\right].$$

Sacando el factor $\sigma(t)$ dentro de la norma,

$$\left\| s_\theta(\mathbf{x}(t), t) + \frac{\mathbf{Z}}{\sigma(t)} \right\|_2^2 = \frac{1}{\sigma^2(t)} \left\| \sigma(t) s_\theta(\mathbf{x}(t), t) + \mathbf{Z} \right\|_2^2 .$$

Por tanto,

$$\mathcal{L}(\theta) = \mathbb{E}_{t,\mathbf{x}_0,\mathbf{Z}} \left[ \frac{\lambda(t)}{\sigma^2(t)} \left\| \sigma(t) s_\theta(\mathbf{x}(t), t) + \mathbf{Z} \right\|_2^2 \right].$$

Si se escoge una ponderación proporcional a la varianza,

$$\lambda(t) \propto \sigma^2(t),$$

entonces el factor $\lambda(t)/\sigma^2(t)$ queda absorbido en una constante de proporcionalidad. Así, el objetivo se simplifica a

$$\mathcal{L}(\theta) \propto \mathbb{E}_{t,\mathbf{x}_0,\mathbf{Z}} \left[ \left\| \sigma(t) s_\theta(\mathbf{x}(t), t) + \mathbf{Z} \right\|_2^2 \right].$$

En la práctica, esta esperanza se estima mediante minibatches de entrenamiento. Dadas muestras $\{\mathbf{x}_i\}_{i=1}^N$, tiempos $t_i \sim U[0,1]$ y ruidos $\mathbf{Z}_i \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$, se construyen muestras perturbadas

$$\mathbf{x}_i(t_i) = \mu_{\mathbf{x}_i}(t_i) + \sigma(t_i)\mathbf{Z}_i.$$

La pérdida empírica usada durante el entrenamiento es entonces

$$\widehat{\mathcal{L}}(\theta) = \frac{1}{N} \sum_{i=1}^{N} \left\| \sigma(t_i) s_\theta(\mathbf{x}_i(t_i), t_i) + \mathbf{Z}_i \right\|_2^2 .$$

Esta es la función de pérdida implementada en el sistema, donde el modelo aprende a predecir el score de las distribuciones perturbadas a partir de imágenes ruidosas y del instante temporal correspondiente.

## I.5. Cálculo de BPD mediante la probability flow ODE

La *probability flow ODE* permite calcular verosimilitudes porque define una transformación determinista entre los datos $\mathbf{x}(0)$ y una variable latente $\mathbf{x}(T)$ cuya distribución es conocida.

La probability flow ODE asociada a una SDE de difusión

$$d\mathbf{x}(t) = \mathbf{f}(\mathbf{x}(t), t)\,\mathrm{d}t + g(t)\,\mathrm{d}\mathbf{W}(t)$$

viene dada por

$$\frac{d\mathbf{x}(t)}{\mathrm{d}t} = \mathbf{f}_{\mathrm{ODE}}(\mathbf{x}(t), t) = \mathbf{f}(\mathbf{x}(t), t) - \frac{1}{2} g^2(t) \nabla_{\mathbf{x}} \log p_t(\mathbf{x}(t)).$$

Como el score real $\nabla_{\mathbf{x}} \log p_t(\mathbf{x})$ no se conoce, se aproxima mediante la red entrenada:

$$\nabla_{\mathbf{x}} \log p_t(\mathbf{x}) \approx s_\theta(\mathbf{x}, t).$$

Por tanto, en la práctica se utiliza

$$\frac{d\mathbf{x}(t)}{\mathrm{d}t} = \mathbf{f}(\mathbf{x}(t), t) - \frac{1}{2} g^2(t) s_\theta(\mathbf{x}(t), t).$$

Al tratarse de una ODE determinista, puede interpretarse como un cambio de variables continuo. La evolución de la log-densidad a lo largo de la trayectoria viene dada por la ecuación de continuidad:

$$\frac{\mathrm{d}}{\mathrm{d}t} \log p_t(\mathbf{x}(t)) = -\nabla_{\mathbf{x}} \cdot \mathbf{f}_{\mathrm{ODE}}(\mathbf{x}(t), t),$$

donde $\nabla_{\mathbf{x}} \cdot$ denota la divergencia respecto a las variables espaciales.

Integrando entre 0 y $T$,

$$\log p_T(\mathbf{x}(T)) - \log p_0(\mathbf{x}(0)) = -\int_0^T \nabla_{\mathbf{x}} \cdot \mathbf{f}_{\mathrm{ODE}}(\mathbf{x}(t), t)\, \mathrm{d}t.$$

Despejando la log-verosimilitud de la muestra inicial,

$$\log p_0(\mathbf{x}(0)) = \log p_T(\mathbf{x}(T)) + \int_0^T \nabla_{\mathbf{x}} \cdot \mathbf{f}_{\mathrm{ODE}}(\mathbf{x}(t), t)\, \mathrm{d}t.$$

Por tanto, para calcular la verosimilitud de una imagen real $\mathbf{x}(0)$ se integra la probability flow ODE hacia adelante desde $t = 0$ hasta $t = T$, acumulando simultáneamente el término de divergencia.

Si al final de la integración $\mathbf{x}(T)$ sigue aproximadamente una distribución gaussiana

$$p_T(\mathbf{x}) = \mathcal{N}(\mathbf{0}, \sigma_T^2 \mathbf{I}),$$

entonces

$$\log p_T(\mathbf{x}(T)) = -\frac{D}{2} \log(2\pi\sigma_T^2) - \frac{1}{2\sigma_T^2} \|\mathbf{x}(T)\|_2^2,$$

donde $D$ es el número total de dimensiones de la imagen.

Una vez calculada la log-verosimilitud, la métrica *bits per dimension* se define como

$$\text{BPD} = -\frac{1}{D \log 2} \log p_0(\mathbf{x}(0)).$$

Para un conjunto de $N$ imágenes, se estima promediando:

$$\text{BPD} = -\frac{1}{ND \log 2} \sum_{i=1}^{N} \log p_0(\mathbf{x}_i).$$

En resumen, el procedimiento es:

1. Tomar una imagen real $\mathbf{x}(0)$.
2. Integrar la probability flow ODE desde $t = 0$ hasta $t = T$.
3. Acumular la divergencia
$$\nabla_{\mathbf{x}} \cdot \mathbf{f}_{\text{ODE}}(\mathbf{x}(t), t).$$
4. Evaluar la densidad final $\log p_T(\mathbf{x}(T))$ bajo la distribución gaussiana conocida.
5. Obtener $\log p_0(\mathbf{x}(0))$ mediante el cambio de variables continuo.
6. Convertir la log-verosimilitud a bits por dimensión.

En la implementación práctica, el cálculo exacto de la divergencia puede ser costoso en alta dimensión, ya que requiere la traza del Jacobiano de la deriva de la ODE. Por ello, se puede aproximar mediante estimadores estocásticos como el estimador de Hutchinson:

$$\nabla_{\mathbf{x}} \cdot \mathbf{f}_{\text{ODE}}(\mathbf{x}, t) = \text{Tr}\left(\frac{\partial \mathbf{f}_{\text{ODE}}}{\partial \mathbf{x}}\right) \approx \mathbf{v}^\top \frac{\partial \mathbf{f}_{\text{ODE}}}{\partial \mathbf{x}} \mathbf{v},$$

donde $\mathbf{v}$ es un vector aleatorio con

$$\mathbb{E}[\mathbf{v}\mathbf{v}^\top] = \mathbf{I}.$$

# Apéndice II. Gráficas complementarias

## II.1. Gráficas de VE

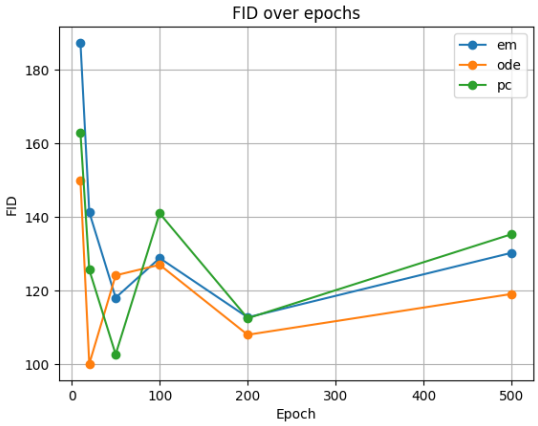



Figura II.1: Evolución del FID para el modelo VE con los samplers propuestos.

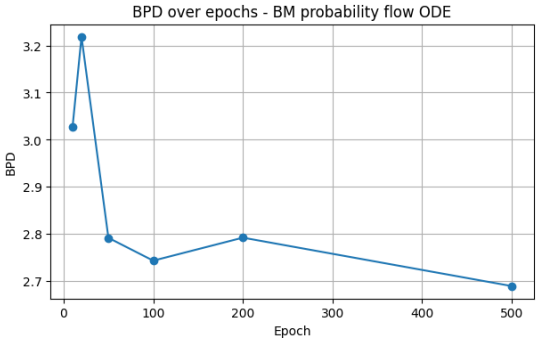



Figura II.2: Bits per dimension (BPD) para el modelo VE.

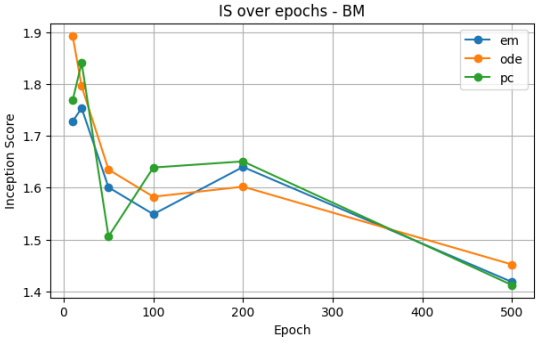
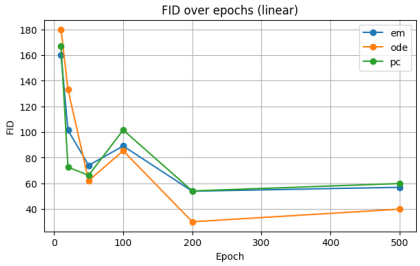
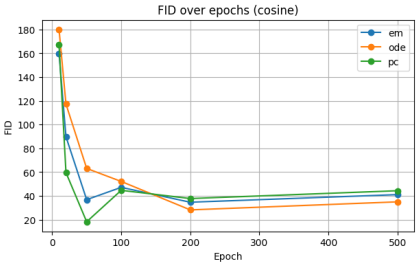



Figura II.3: Inception Score (IS) para el modelo VE.

## II.2. Gráficas de VP




(a) Linear




(b) Cosine




(c) Sigmoid

Figura II.4: Comparación de schedules en VP mediante FID.

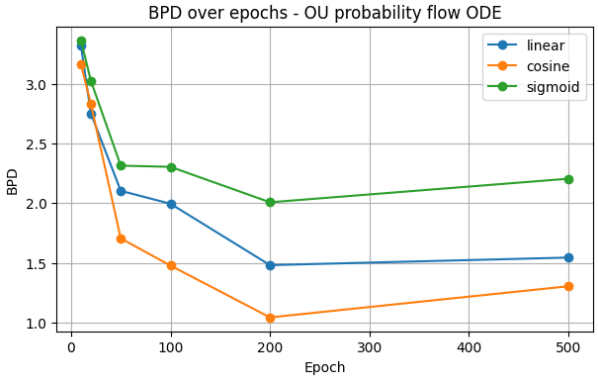
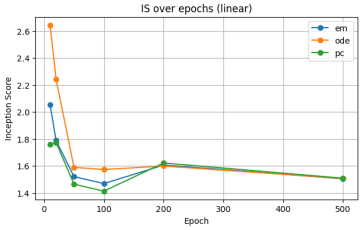
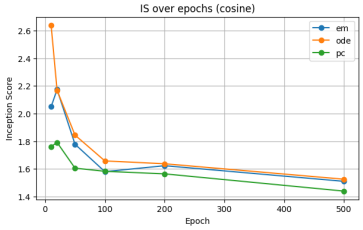


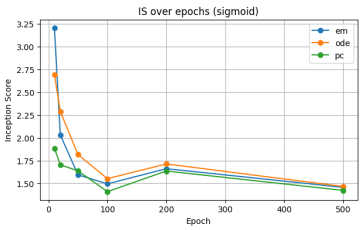

Figura II.5: Bits per dimension (BPD) para los distintos *schedules* del modelo VP.




(a) Linear




(b) Cosine




(c) Sigmoid

Figura II.6: Inception Score (IS) para los distintos *schedules* del modelo VP.

## II.3. Gráficas comparativas entre VE y VP

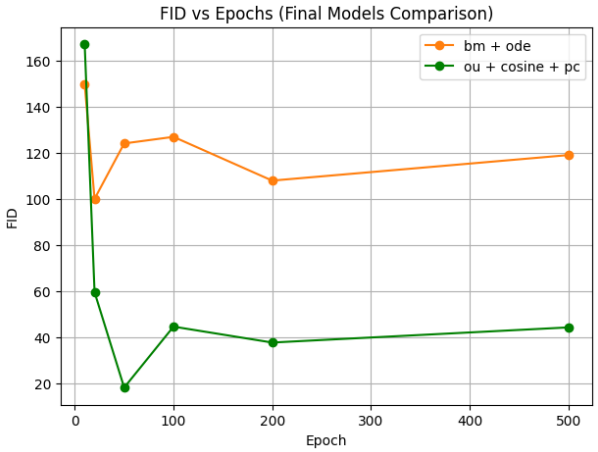



Figura II.7: Comparación de FID entre los modelos VE y VP.

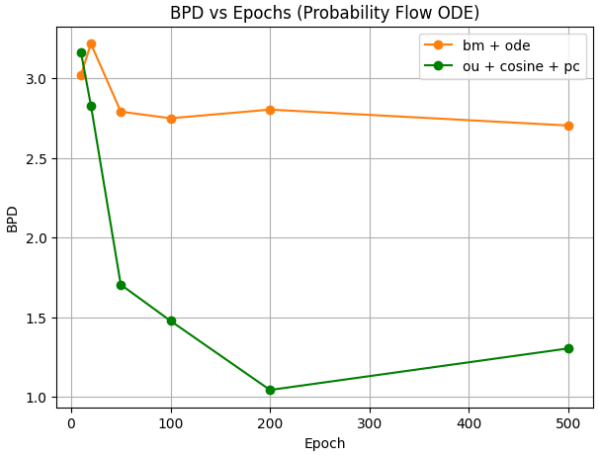



Figura II.8: Comparación de BPD entre los modelos VE y VP.

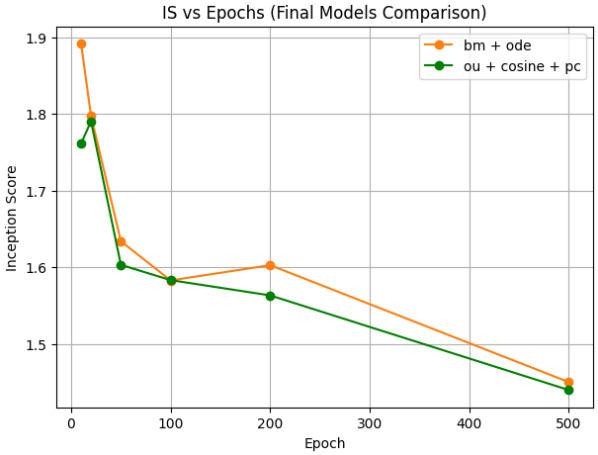



Figura II.9: Comparación de Inception Score (IS) entre los modelos VE y VP.

# Apéndice III. Generación de imágenes en AFHQ

En este apéndice se presenta una extensión del sistema desarrollado aplicada al dataset AFHQ, centrado en imágenes de gatos. A diferencia de MNIST, este dataset presenta una complejidad significativamente mayor, debido a la variabilidad en color, textura, iluminación y estructura de las imágenes.

Para adaptar el modelo a este dominio, se utilizaron únicamente las imágenes pertenecientes a la clase *cat*. Las imágenes originales del dataset AFHQ tienen una resolución de $512 \times 512$ píxeles; sin embargo, debido a las limitaciones computacionales y con el objetivo de hacer viable el entrenamiento, se redimensionaron a $64 \times 64$ píxeles. Esta reducción de resolución supone una pérdida significativa de información espacial, especialmente en detalles finos y texturas, lo que limita la capacidad del modelo para capturar estructuras de alta frecuencia. Posteriormente, las imágenes se transformaron a tres canales RGB y se aplicó una normalización al intervalo $[-1, 1]$ mediante medias y desviaciones típicas $(0{,}5, 0{,}5, 0{,}5)$, con el objetivo de mantener una escala coherente con la utilizada durante el entrenamiento de los modelos de difusión.

La Figura III.1 muestra ejemplos reales del subconjunto utilizado. Se aprecia una gran variabilidad en la pose, el color, la iluminación, la textura y el fondo, lo que explica la mayor dificultad del experimento en comparación con MNIST.



Figura III.1: Ejemplos de imágenes reales del subconjunto de gatos de AFHQ.

El modelo empleado fue una red de score basada en U-Net adaptada a imágenes RGB, con tres canales de entrada y salida. Para aumentar su capacidad respecto a los experimentos sobre MNIST, se utilizaron canales $(64, 128, 256, 512)$ y un embedding temporal de dimensión 256. El proceso de difusión seleccionado fue el modelo VP basado en Ornstein–Uhlenbeck con *schedule* cosine, manteniendo la configuración que había mostrado mejor comportamiento en los experimentos principales.

Inicialmente, el modelo se entrenó durante 1000 épocas con una tasa de aprendizaje de $10^{-4}$, guardando checkpoints intermedios en las épocas 10, 20, 50, 100, 200, 500 y 1000. Tras observar que las muestras generadas comenzaban a mostrar estructuras visuales propias de gatos, pero todavía presentaban una calidad limitada, se decidió continuar el entrenamiento desde el checkpoint de la época 1000. En esta segunda fase se redujo la tasa de aprendizaje a $10^{-5}$ y se entrenó hasta la época 2000, almacenando checkpoints adicionales en las épocas 1100, 1200, 1500 y 2000.

Se generaron 20 imágenes finales, empleando $T = 1{,}0$, $t_{\text{end}} = 10^{-3}$ y 1000 pasos de integración. El proceso de muestreo se inicializó con ruido gaussiano escalado por $\sigma(T)$ y se utilizó un único paso corrector por iteración, con un valor de $SNR$ igual a 0.03. Esta configuración se mantuvo fija entre experimentos, de modo que las mejoras observadas pueden atribuirse principalmente a la mejora en el entrenamiento del modelo.

La Figura III.2 muestra la evolución del proceso de generación para el modelo entrenado durante 2000 épocas, utilizando el sampler predictor–corrector. Se observa cómo el modelo reduce progresivamente el ruido inicial y comienza a formar estructuras reconocibles, como los contornos de la cabeza, zonas de contraste asociadas a los ojos y patrones de color compatibles con imágenes de gatos. La visualización se presenta únicamente para este caso final, ya que permite ilustrar el comportamiento del modelo en su estado más refinado, siendo las trayectorias de etapas anteriores cualitativamente similares pero menos definidas.

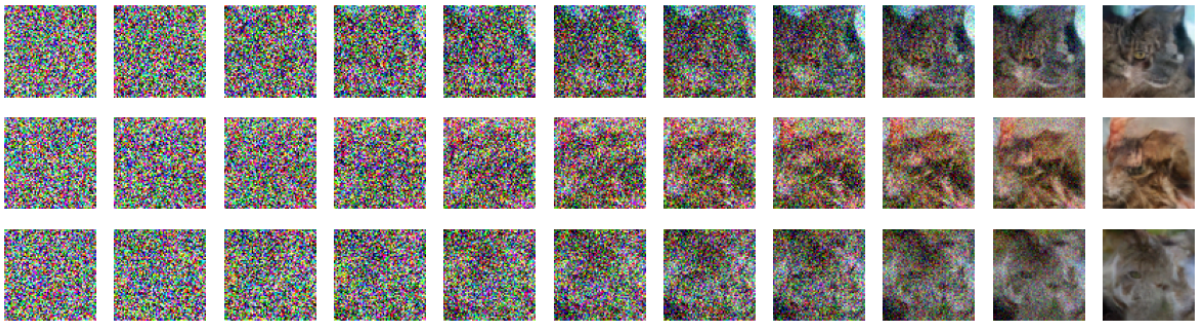


Figura III.2: Evolución del proceso de generación mediante el sampler predictor–corrector en el dataset AFHQ para 2000 épocas y 1000 pasos de integración.

La Figura III.3 compara las muestras generadas tras 1000 y 2000 épocas de entrenamiento. En la primera fase, el modelo ya comienza a capturar estructuras generales, pero las muestras todavía aparecen difusas y con escasa coherencia semántica. Tras continuar el entrenamiento hasta 2000 épocas con una tasa de aprendizaje menor, se observa una mejora clara en la coherencia global, una distribución de color más estable y una mayor definición de rasgos característicos como ojos, hocico y contornos de la cabeza. No obstante, persisten limitaciones en forma de falta de nitidez, deformaciones estructurales y dificultades para mantener relaciones espaciales consistentes entre dichos rasgos.



(a) Muestras tras 1000 épocas.



(b) Muestras tras 2000 épocas.

Figura III.3: Comparación cualitativa de imágenes generadas en AFHQ antes y después de la segunda fase de entrenamiento.

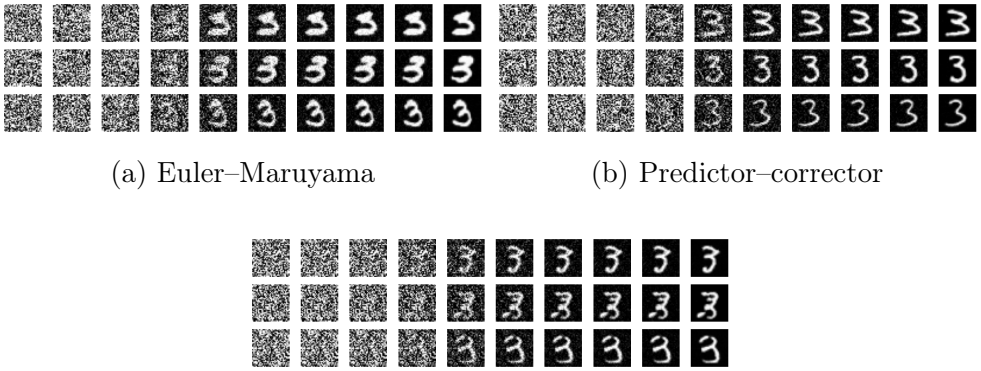
Estas limitaciones son coherentes con la mayor complejidad del dataset y con las restricciones computacionales del experimento. Aunque el aumento en el número de épocas produjo una mejora notable, se decidió no continuar entrenando el modelo debido al coste en tiempo y a la disponibilidad limitada de GPU. Asimismo, la reducción de resolución de $512 \times 512$ a $64 \times 64$ contribuye a la pérdida de detalle y a la suavización de las texturas observadas en las muestras generadas.

Además de los factores anteriores, es importante señalar que la arquitectura U-Net empleada, aunque adecuada para tareas de generación en dominios de baja y media complejidad, puede presentar limitaciones en escenarios como AFHQ. En particular, el uso de una arquitectura relativamente compacta y con un número limitado de niveles puede dificultar la captura de dependencias de largo alcance y relaciones globales en la imagen. Esto puede dar lugar a incoherencias estructurales entre distintas regiones, como la posición relativa de los ojos o la forma global de la cabeza. Asimismo, el tamaño reducido de las representaciones intermedias en las capas más profundas puede actuar como un cuello de botella, limitando la capacidad del modelo para preservar información semántica compleja a lo largo del proceso de generación.

Como línea futura, sería interesante explorar arquitecturas más profundas o con mecanismos de atención, así como el uso de resoluciones más altas, con el fin de mejorar la captura de dependencias globales y la generación de detalles finos.

Por tanto, estos resultados deben interpretarse como una validación del pipeline en imágenes RGB y como una prueba de escalabilidad del sistema, más que como un resultado competitivo en generación de imágenes naturales. El experimento confirma que la arquitectura implementada puede extenderse a imágenes en color, aunque también evidencia que datasets como AFHQ requieren modelos de mayor capacidad, más datos efectivos y entrenamientos más prolongados para alcanzar una calidad visual comparable a la obtenida en dominios más simples.

# Apéndice IV. Imágenes generadas adicionales

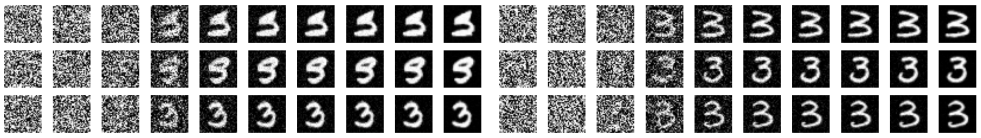


(a) Euler–Maruyama (b) Predictor–corrector

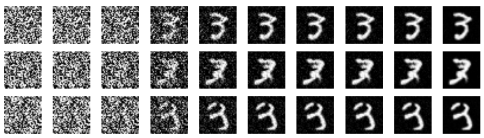


(c) Probability Flow ODE

Figura IV.1: Comparación de los samplers para el modelo VP con schedule linear.



(a) Euler–Maruyama (b) Predictor–corrector



(c) Probability Flow ODE

Figura IV.2: Comparación de los samplers para el modelo VP con schedule sigmoid.